

PROJECT BRIEF

PRISMATICA

Like a prism refracts light into its components, Prismatica breaks raw data into structured views, dashboards, and live adapters — tailored to every professional workflow.

DATE

21 February 2026

TYPE

Educational — Not for redistribution

PRISMATICA

Professional Polymorphic Data Platform — Project Brief

Like a prism refracts light into its components, Prismatica breaks raw data into structured views, dashboards, and live adapters — tailored to every professional workflow.

Table of Contents

1. [Project Overview](#)
2. [Competencies Assessed](#)
3. [Company Context](#)
4. [Project Description](#)
5. [Core Concept — Polymorphic Data Architecture](#)
6. [Functional Requirements](#)
 - 6.1 [Database Builder](#)
 - 6.2 [View System \(Polymorphic Views\)](#)
 - 6.3 [Adapter Layer](#)
 - 6.4 [Dashboard Builder](#)
 - 6.5 [Embed & External Impact](#)
 - 6.6 [Authentication & Accounts](#)
 - 6.7 [User Space](#)
 - 6.8 [Employee Space](#)
 - 6.9 [Administrator Space](#)
 - 6.10 [Contact Page](#)
7. [Business Rules & Constraints](#)
8. [Security & GDPR](#)
9. [Accessibility](#)
10. [Technical Stack](#)
11. [Deliverables](#)
12. [Annex 1 — Relational Database Schema \(indicative MCD\)](#)
13. [Annex 2 — Key Use Cases](#)
14. [Annex 3 — Polymorphic Architecture Overview](#)

1. Project Overview

Field	Value
Project name	Prismatica
Type	Polymorphic Data Platform / Professional Dashboard Tool
Estimated duration	70 hours
Authorised resources	Annexes, official documentation
Submission format	Public GitHub repository + deployed URL + project management link

2. Competencies Assessed

All competencies of the **Web & Web Mobile Developer** professional qualification are evaluated through this project.

Activity Type 1 — Front-end development of a secure web application:

- Install and configure the work environment according to the project
- Design and prototype user interfaces (wireframes and mockups)
- Build static and responsive interfaces
- Develop dynamic, interactive front-end components (schema builder, drag-and-drop layout, live charts)

Activity Type 2 — Back-end development of a secure web application:

- Set up a relational database (schema creation, migrations, seeding)
- Develop SQL and NoSQL data access components
- Design a REST (or GraphQL) API to serve the polymorphic adapter layer
- Document and deploy the full application to a production environment

3. Company Context

NovaSphere is a consulting firm of seven developers and designers. They work with clients ranging from logistics companies, marketing agencies, to independent restaurant chains — all of them sharing the same pain point: their data lives in spreadsheets, disconnected tools, or proprietary software with no visual interface they actually own.

NovaSphere's team has been manually building custom dashboards for each client for years. It is no longer sustainable. They have decided to invest in building **Prismatica** — an internal product that any professional can use to:

- **Own their data** by creating and managing their own database schemas directly in the tool
- **Define how they see it** through a polymorphic view system (tables, charts, KPIs, calendars...)
- **Connect it to their world** via an adapter layer that pushes or pulls data to/from external systems
- **Share it professionally** through embeddable widgets and public/private dashboards

You have been hired by **DevForge**, a development agency contracted by NovaSphere, and you are now assigned to this project.

4. Project Description

Your project manager has gathered requirements from NovaSphere's team and translates them for you as follows.

The goal of Prismatica is not just to *display* data — it is to let a professional user **model their data, structure it, query it visually, and surface it anywhere they need**, without writing a single line of SQL or code. The key differentiator from existing tools (Airtable, Notion, Metabase, Grafana) is the **polymorphic adapter system**: a single data model can simultaneously power a dashboard widget, a live feed on a client's website, a filtered export, and a REST endpoint — all derived from the same underlying schema and kept in sync.

5. Core Concept — Polymorphic Data Architecture

This is the philosophical and technical foundation of the entire project. Before reading the functional requirements, your team must understand this model.

5.1 The Three Layers

LAYER 1 – DATA MODEL (Schema)

The user defines their own database: tables, columns, types, relations. Prismatica stores this in a secure, managed schema on the platform's servers.

Example: A restaurant owner creates a "Reservations" table with fields: date, name, party_size, status, special_requests



LAYER 2 – VIEWS (Polymorphic)

The user creates multiple views on the same data – each view is a different representation: a bar chart, a calendar, a KPI tile, a filterable table, a map. Views are reusable and composable.

Same "Reservations" data → Timeline view + KPI "total tonight" + Bar chart "bookings per day" + Table with inline edit



LAYER 3 – ADAPTERS (Output / Input connectors)

Adapters connect the views or raw data to the outside world. An adapter can be an embed script, a REST endpoint, a webhook, a CSV export, a live feed, or a read/write sync with an external API.

Same "Reservations" data →

- Embed on restaurant website (live availability widget)
- REST endpoint GET /api/reservations (for mobile app)
- Webhook to notify staff when new booking arrives
- CSV export every Monday at 8am

5.2 Why "Polymorphic"?

The same underlying data model produces **multiple forms** depending on context: a view is not tied to one output. This is the polymorphic contract: any view can be mounted in a dashboard, embedded externally, or exposed as an adapter endpoint — without duplicating data or logic. Your architecture must reflect this: views and adapters must be loosely coupled to the data model through a clean interface layer (think strategy pattern or plugin architecture on the back end).

6. Functional Requirements

6.1 Database Builder

This is the entry point for any new project in Prismatica. A user creates a **Project**, and inside a project they define one or more **Collections** (the equivalent of database tables).

A Collection has the following characteristics:

- A unique name within the project (e.g. "Products", "Invoices", "Site Visits")
- A list of **Fields**, each with:
 - A name and a display label
 - A type: `text`, `number`, `boolean`, `date`, `datetime`, `email`, `url`, `select` (enum), `relation` (foreign key to another collection), `file` (attachment), `computed` (formula based on other fields)
 - Validation rules per field: required, min/max, regex, uniqueness
 - Default value
- A primary key (auto-generated UUID by default, or custom)
- Optional: soft-delete support (adds a `deleted_at` field and never hard-deletes rows)
- Optional: audit trail (auto-tracks `created_at`, `updated_at`, `created_by`, `updated_by`)

The schema builder interface must:

- Allow creating, editing, and deleting collections and fields through a visual drag-and-drop interface
- Show a live preview of the schema as a diagram (entity-relationship style)
- Detect and warn about breaking changes (e.g. deleting a field that is used in a view)
- Support adding relations between collections (one-to-one, one-to-many, many-to-many via pivot)
- Allow importing a schema from a CSV header row (auto-detect column names and suggest types)
- Allow exporting the schema as a SQL file or JSON schema

Data entry and management:

- Once a collection is defined, the user can enter data through a spreadsheet-style inline editor
- Bulk import from CSV or JSON
- Filtering, sorting, and searching rows directly in the collection editor
- Row-level actions: edit, duplicate, delete, view history (if audit trail enabled)

6.2 View System (Polymorphic Views)

A **View** is a named, configurable representation of data from one or more Collections. Views are the central object of the system — they are the unit that gets placed on dashboards and exposed through adapters.

Available view types (minimum required):

View Type	Description
Table View	Paginated, sortable, filterable spreadsheet of rows. Supports inline editing.
Bar Chart	Vertical or horizontal. Supports grouping and stacking.
Line Chart	Single or multi-series. Supports time-axis with smart tick spacing.
Pie / Donut Chart	For proportional data. Configurable legend position.
Scatter Plot	Two numeric axes, optional colour grouping by a third field.
KPI Tile	Single computed value (sum, count, average, min, max) with label, unit, trend indicator, and configurable threshold colours (green / orange / red).
Calendar View	Renders rows as events on a monthly/weekly calendar using a date field.
Kanban View	Renders rows as cards grouped by a <code>select</code> field. Cards are draggable between columns to update status.
Map View	Renders rows on a geographic map using <code>lat</code> / <code>lng</code> fields or address geocoding. (<i>Advanced — optional bonus</i>)

Each view is configured with:

- A name and optional description
- Source collection(s) — a view can join two related collections
- Field mappings (which field maps to which axis / dimension / label)
- Aggregation function per metric field: `sum` , `avg` , `count` , `min` , `max` , `count_distinct`
- A filter set: one or more conditions combined with AND/OR logic
- A sort order
- A refresh strategy: `manual` , `every 10s` , `every 30s` , `every 1min` , `every 5min`
- Visibility: `private` (owner only), `workspace` (all project members), `public` (embed-accessible)
- Colour theme: predefined palettes or custom hex values per series

A view must be re-renderable without reloading the page when its data source updates.

6.3 Adapter Layer

Adapters are the connectors between Prismatic's internal data and the outside world. Each adapter is linked to a view or a collection and defines how data flows in or out.

Supported adapter types:

Outbound adapters (Prismatic → Outside):

- **Embed Script:** generates a `<script>` tag and optionally an `<iframe>` that renders a view on any external website in read-only mode. The embed respects the view's filter and refresh settings. Theming (light/dark, accent colour) is configurable via URL parameters.

- **REST Endpoint:** exposes a collection's data (filtered by the view's conditions) as a JSON API endpoint. Supports `GET` only (read-only by default), with optional API key protection. Pagination via `?page=` and `?limit=` parameters.
- **Webhook (Push):** when a row in a collection is created, updated, or deleted, Prismatic sends a POST request to a configured URL with the row payload. Configurable per event type.
- **Scheduled CSV Export:** generates a CSV of the view's data on a configurable schedule (daily, weekly, monthly) and sends it to a configured email address or deposits it in a configured FTP/S3 path.
(Advanced — optional)

Inbound adapters (Outside → Prismatic):

- **REST Write Endpoint:** exposes a collection as a writable API endpoint (POST / PATCH / DELETE) protected by an API key. Enables external apps to push data into Prismatic collections.
- **CSV Import Schedule:** Prismatic polls a given URL at a defined interval, downloads a CSV, and upserts rows into a collection based on a configurable unique key field.
- **Form Endpoint:** generates a hosted form URL that non-authenticated users can visit to submit a row into a collection (e.g. a contact form, a survey, a booking request). The form fields and validation are derived from the collection schema.

Adapter management interface must:

- List all active adapters per project with status (active, error, paused)
- Show the last execution time and result (success / error + message) for scheduled/webhook adapters
- Allow pausing and resuming adapters individually
- Allow regenerating API keys without deleting the adapter configuration
- Show a live log of the last 50 adapter executions

6.4 Dashboard Builder

A **Dashboard** is a named canvas that organises multiple Views into a coherent interface.

A Dashboard has:

- A title and optional description
- A workspace (belongs to a project)
- A visibility setting: `private`, `team`, or `public` (generates a shareable read-only URL)
- A grid layout: views are placed as resizable, draggable widgets on a 12-column grid
- Global filter controls: one or more filter inputs at the dashboard level that propagate to all compatible views (views that share the filtered field)
- A global refresh toggle: force all views to refresh simultaneously
- A last-modified timestamp and author

The dashboard layout must:

- Be fully drag-and-drop within the builder interface
- Support widget resize handles (width and height independently)

- Persist the layout automatically after every change (debounced, max 2s delay)
- Render correctly at desktop ($\geq 1280\text{px}$), tablet ($768\text{--}1279\text{px}$), and mobile ($< 768\text{px}$) screen sizes. On mobile, widgets stack vertically in a configurable order.
- Allow adding a **text/markdown block** as a non-data widget (for headings, notes, instructions)
- Allow adding a **divider** widget

Dashboard sharing:

- A public dashboard link must be accessible without authentication
- The link can optionally be protected by a simple access password (not requiring account creation)
- The public view must be fully read-only — no schema or data modification is possible from a public link
- The owner can revoke a public link at any time (generates a new token, old link becomes invalid)

6.5 Embed & External Impact

This section describes how Prismatica connects to the outside world through its embed system.

Embed script behaviour:

- The embed script fetches the view data via the adapter REST endpoint at the configured refresh interval
- Renders the view using the same rendering engine as the main app (consistent visual output)
- Supports a `theme` URL parameter: `light`, `dark`, or `auto` (follows the host page's `prefers-color-scheme`)
- Supports a `locale` URL parameter for number and date formatting
- Must not interfere with the host page's CSS or JavaScript (scoped styles, Shadow DOM or iframe isolation)
- Must display a loading state and a graceful error state if the endpoint is unreachable

Impact on external websites:

- A client embeds a Prismatica KPI tile showing live stock count on their e-commerce product page
- A restaurant embeds a Prismatica calendar view showing available reservation slots on their booking page
- A marketing team embeds a Prismatica bar chart on their internal intranet to display campaign performance

All of the above must work from a single embed snippet, with no external dependency other than the Prismatica CDN-hosted script.

6.6 Authentication & Accounts

Account creation (self-registration):

A visitor can create an account by providing:

- First name and last name

- Email address (used as login identifier, must be unique)
- A secure password: minimum 12 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character

Upon account creation, the role **"user"** is assigned by default. A welcome email is sent automatically.

Login:

The user logs in with their email and password. A "Forgot password" flow must be available: the user provides their email, receives a time-limited reset link (valid for 30 minutes), and follows it to define a new password.

Session security:

- Sessions must expire after a configurable period of inactivity (default: 24 hours)
- A user can view and revoke all active sessions from their profile
- Failed login attempts must be rate-limited (lockout after 5 failed attempts within 10 minutes)

6.7 User Space

From their personal space, an authenticated user can:

- **Projects:** create, rename, archive, and delete projects. A project is the top-level container (holds collections, views, dashboards, adapters).
- **Collections:** build and manage database schemas within a project (see 6.1)
- **Views:** create and configure views on their data (see 6.2)
- **Dashboards:** build and share dashboards (see 6.4)
- **Adapters:** manage input/output connectors (see 6.3)
- **Profile:** edit personal information (name, email, password, profile picture)
- **Sessions:** view and revoke active sessions
- **Data deletion:** request full account and data deletion (GDPR right to erasure). This action triggers a confirmation email and is irreversible after 7 days.

6.8 Employee Space

An employee, after login, has access to all capabilities of a standard user, plus:

- **Project overview:** can see all projects across all users of the platform (read-only unless granted explicit access)
- **Adapter monitoring:** can see the status of all adapters platform-wide, restart failing adapters
- **Public dashboard moderation:** can unpublish a public dashboard that contains inappropriate content, with a mandatory reason field. The owner is notified by email.
- **User management (read-only):** can view the list of all registered users, their account status, and their project count

- **Platform usage statistics:** number of active users, collections created, rows stored, adapter executions per day — displayed as a set of KPI tiles and charts

An employee cannot modify or cancel a user's data without explicit documented reason (must log contact method and reason, similar to the audit trail pattern).

6.9 Administrator Space

The administrator has access to all employee capabilities, plus:

User management (full):

- Create employee accounts: provide email + temporary password. The employee receives a notification email that their account has been created, but the password is **not** included in the email — they must obtain it directly from the administrator.
- Activate and deactivate user or employee accounts (soft-disable: the account persists but cannot log in)
- Permanently delete user accounts (with all associated data) on explicit request
- **Note:** it must not be possible to create an administrator account from within the application interface. The initial admin account must be seeded directly in the database.

Platform analytics (NoSQL-powered):

- View the number of projects, collections, views, and dashboards created per user — rendered as a comparison chart
- View adapter execution volume and error rate over time (time-series chart)
- View the most-used view types across the platform (pie chart)
- All analytics data must be stored in and queried from a **non-relational database** (MongoDB or equivalent)
- A revenue/usage report per subscription tier must be available with filters by date range and plan

System configuration:

- Configure global settings: max rows per collection per free-tier user, max adapters per project, allowed file upload types and sizes
- View and manage the platform's email templates (welcome, password reset, adapter error notification)

6.10 Contact Page

Any visitor can access a contact form from the main navigation. The form collects:

- A subject / title
- A message body
- The sender's email address

On submission, the message is forwarded by email to the NovaSphere support team. A confirmation message is shown to the visitor. The form must be protected against spam (honeypot field or CAPTCHA).

7. Business Rules & Constraints

7.1 Scalability of Views

- All chart and table views must be rendered in SVG or Canvas — no static image output
- When a collection exceeds 10,000 rows, views must apply server-side aggregation before sending data to the client. The raw rows must never be sent to the browser in bulk.
- When a view is resized within a dashboard, it must re-render responsively without triggering a new data fetch
- A view's configuration change must immediately trigger a data refresh and re-render
- Axis labels, legends, and tooltips must adapt dynamically to the available widget size (truncation + full value on hover)

7.2 Schema Modification Rules

- Adding a new field to a collection is non-destructive and always allowed
- Renaming a field updates all references (views, adapters, filters) automatically
- Deleting a field that is referenced by one or more views or adapters must be blocked until those references are removed, with a clear error listing the impacted views/adapters
- Changing a field's type is allowed only if the new type is compatible with existing data (e.g. `text` → `email` is fine if all values match email format; `number` → `text` is always allowed; `text` → `number` requires validation and is blocked if any existing value is non-numeric)
- Deleting a collection follows the same blocking logic: cannot be deleted if it is the source of an active view or adapter

7.3 Adapter Rules

- A REST endpoint adapter generates a unique, opaque URL (e.g. `/api/v1/adapt/a3f9b2c1...`) that does not expose the collection name
- API keys for adapters must be stored hashed and are displayed only once at creation
- A webhook adapter must implement retry logic: on failure (non-2xx or timeout), retry 3 times with exponential backoff (1s, 4s, 16s)
- An embed script adapter must respect CORS: only allow requests from the domains registered by the user during adapter configuration

7.4 Dashboard Rules

- A dashboard may contain at most 24 widgets (views + text blocks) to maintain performance
- Layout is saved automatically with a debounce of 2 seconds after the last interaction
- A public dashboard link uses a token with no expiry unless explicitly revoked

- When a user deletes a view that is placed on a dashboard, the dashboard widget must display a graceful "View no longer available" placeholder rather than breaking

8. Security & GDPR

- Passwords must be stored hashed using **bcrypt** (cost factor ≥ 12) or **argon2id**
- All API routes must be protected by authentication (JWT with short-lived access token + refresh token rotation, or secure server-side sessions)
- All user data is scoped to the authenticated user — cross-user data access must be impossible at the query level (use row-level security or equivalent middleware guards)
- All user-uploaded files must be scanned for type (magic bytes, not extension only) before storage
- Collection data entered by users is stored in an isolated schema or namespace per user/project to prevent SQL injection and cross-tenant leakage
- The application must implement rate limiting on all public endpoints (authentication, contact form, public dashboard, embed endpoint)
- Sensitive environment variables (DB credentials, SMTP credentials, JWT secret, API keys) must never be committed to the repository. Use `.env` files with a `.env.example` template.
- Users must be able to export all their data as a ZIP (collections as CSV, views and dashboard configs as JSON) — GDPR right to data portability
- Users must be able to request full deletion of their account and all associated data — GDPR right to erasure
- The application must display a clear cookie/privacy notice on first visit. Session cookies must be `HttpOnly` and `Secure`.

9. Accessibility

The application must comply with **WCAG 2.1 Level AA** requirements:

- All interactive elements must be keyboard-navigable (tab order, focus visible)
- All form fields must have explicit `<label>` elements or `aria-label` attributes
- All charts and data visualisations must have a textual alternative (either a data table toggle or an `aria-describedby` summary)
- Colour contrast ratio must meet 4.5:1 for normal text and 3:1 for large text
- Error messages must be programmatically associated with their input field (`aria-describedby`)
- Drag-and-drop interactions in the dashboard and schema builder must have keyboard-accessible alternatives

- Dynamic content updates (view refresh, dashboard save indicator) must announce changes via `aria-live` regions

10. Technical Stack

No technology is imposed for this project, with the exception that **both a relational and a non-relational database must be used**. The following is a suggested stack for reference:

Layer	Suggested Technologies
Front-end framework	React (Vite) or Vue 3 (Nuxt)
UI / Styling	Tailwind CSS or shadcn/ui
Chart rendering	ECharts, Recharts, or D3.js
Drag-and-drop (dashboard)	react-grid-layout or dnd-kit
Drag-and-drop (schema builder)	dnd-kit or @dnd-kit/sortable
Back-end	Node.js (Express / Fastify) or PHP (Laravel / Symfony)
Relational database	PostgreSQL (recommended), MySQL, or MariaDB
Non-relational database	MongoDB (analytics layer) or Redis
Authentication	JWT (access + refresh token) or Passport.js sessions
File storage	Local (dev), S3-compatible (prod — AWS S3, MinIO, Cloudflare R2)
Email	NodeMailer + SMTP, Resend, or Mailgun
Deployment	Fly.io, Railway, Render, Vercel (front) + PlanetScale / Supabase

All technical choices must be **justified in your technical documentation** — explain why you chose your stack and what alternatives you considered.

11. Deliverables

11.1 Required Links (to submit)

- Public GitHub repository URL (or URLs if front/back are separate)
- Deployed application URL (functional and accessible at delivery time)
- Project management tool URL (Jira, Notion, Linear, Trello, etc.) — must be accessible to the jury

11.2 Git Repository Contents

Your repository must contain:

README.md — Installation guide:

- Prerequisites and local environment setup steps
- Environment variable reference (`.env.example` file)
- Database setup and seeding commands
- How to run tests
- Test credentials for each role (visitor, user, employee, admin)

Git practices:

- `main` branch — production-ready code only
- `develop` branch — integration branch
- Feature branches: `feature/[feature-name]` branched from `develop`, merged back via pull request after testing
- Commit messages must follow Conventional Commits format: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`

Database files:

- SQL schema creation file (`schema.sql` or migration files)
- SQL seed file with representative test data (`seed.sql`)
- MongoDB seed scripts (if applicable)
- Note: using an ORM migration tool does not substitute for having raw SQL files in the repository

11.3 PDF Documentation Package

A. Graphic Charter (`charte-graphique.pdf`):

- Colour palette (primary, secondary, accent, neutral, semantic colours) with hex codes
- Typography choices (font families, size scale, weight usage)
- Component library overview (buttons, inputs, cards, badges)
- Wireframes: 3 desktop wireframes + 3 mobile wireframes (key screens)
- Mockups: 3 desktop mockups + 3 mobile mockups (high-fidelity)

B. User Manual (`manuel-utilisateur.pdf`):

- Overview of the application and its main workflows
- Step-by-step walkthroughs for each user role
- Test credentials for each role
- Known limitations and edge cases

C. Technical Documentation (`documentation-technique.pdf`):

- Initial technology research and stack justification
- Local environment configuration guide
- Conceptual Data Model (MCD) or UML Class Diagram
- Use case diagram
- Sequence diagrams for at least 3 key flows (e.g. user login, view creation, adapter execution)
- Polymorphic architecture design explanation (how the 3-layer model is implemented)
- Deployment procedure (step-by-step, reproducible)
- Security measures implemented

D. Project Management Report ([gestion-projet.pdf](#)):

- Methodology used (Scrum, Kanban, etc.) and justification
- Sprint breakdown or task decomposition
- Time tracking summary
- Retrospective: what went well, what was challenging, what you would do differently

Annex 1 — Relational Database Schema (indicative MCD)

The following schema is provided as a starting point. You are free to extend, adapt, or restructure it based on your architectural decisions. All choices must be justified in your technical documentation.



Note: Collection rows (the actual user data) are stored in a dynamically generated schema per project (e.g. PostgreSQL schema per project, or a [collection_rows](#) EAV table, or a hybrid approach). Your technical documentation must justify how you handle row storage for dynamic schemas.

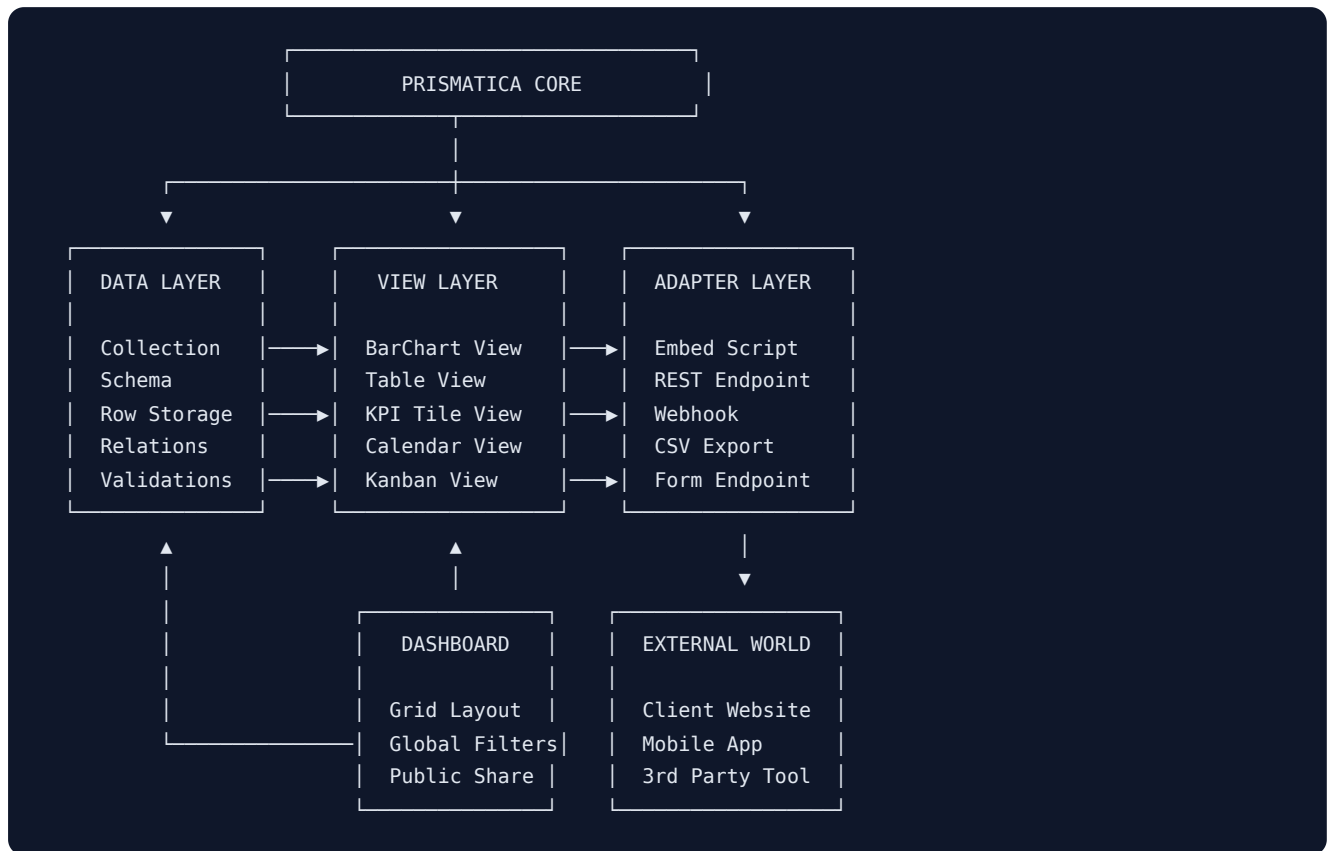
Annex 2 — Key Use Cases

The following use cases are the minimum expected in your documentation. You may add more.

ID	Use Case	Primary Actor
UC-01	Register on the platform	Visitor
UC-02	Log in / Log out	User, Employee, Admin
UC-03	Reset password via email link	User
UC-04	Create a project	User
UC-05	Define a collection schema (add fields, set types, add relations)	User
UC-06	Import data into a collection via CSV	User
UC-07	Create a view on a collection	User
UC-08	Configure view filters and aggregations	User
UC-09	Create a dashboard and add views as widgets	User
UC-10	Rearrange and resize dashboard widgets via drag-and-drop	User
UC-11	Apply a global filter across a dashboard	User
UC-12	Share a dashboard via a public link	User
UC-13	View a public dashboard without an account	Visitor
UC-14	Create an embed adapter and get the embed snippet	User
UC-15	Embed a live view on an external website	External User / Visitor
UC-16	Create a REST endpoint adapter for a collection	User
UC-17	Configure a webhook adapter for row events	User
UC-18	Moderate (unpublish) a public dashboard	Employee
UC-19	Create an employee account	Admin
UC-20	Deactivate a user or employee account	Admin
UC-21	View platform analytics (NoSQL)	Admin
UC-22	Rename a field and see all view references update	User
UC-23	Delete a field blocked by active view reference	User (blocked)
UC-24	Request full account and data deletion (GDPR)	User
UC-25	Export all personal data as ZIP (GDPR)	User
UC-26	Submit a contact form	Visitor
UC-27	Revoke a public dashboard link	User
UC-28	View and revoke active login sessions	User

Annex 3 — Polymorphic Architecture Overview

The following diagram illustrates the expected relationship between the three core layers and how they interact at runtime.



Key architectural principle: a View must implement a single rendering interface regardless of where it is mounted (dashboard widget, embed, public URL). An Adapter must be able to wrap any View or Collection through the same connector interface. This is the polymorphic contract your back-end and front-end architectures must honour.

© DevForge — Reproduction restricted. This document is intended for the jury of the professional qualification TP Développeur Web et Web Mobile.